

WiFi-Proximity-Based Classroom Attendance: A Privacy-Preserving Captive Portal System on Commodity Android Hardware

Vivek Barhate

Department of Computer Engineering
barhatevisb@gmail.com

Abstract

Taking attendance in class is still mostly done by reading out names, and the digital replacements I tried (Google Forms, RFID cards, a Bluetooth-beacon app) each had a different way of being cheated. This report describes an Android app I built that uses the teacher’s own phone as a small WiFi access point: students join it, scan a QR code, and type their roll number into a captive page served by an HTTP server running on that same phone. To make sure a student doesn’t just check in and walk out, the app keeps watching — the teacher phone pings everyone on the subnet every five seconds, and the student’s browser pings the teacher every thirty seconds. Every stored identifier is a SHA-256 hash of the student’s IP, salted with a UUID that is fresh for each session, so the records can’t be linked across classes. The whole system runs offline on a stock Android phone (API 26 and up) with no backend at all. Along the way I describe the bugs I hit — most embarrassingly, a single test phone showing up as 242 “connected students” — and the things WiFi proximity is and isn’t good for.

Keywords: attendance, WiFi, captive portal, Android, privacy, hashing, proximity sensing.

1 Introduction

Roll-call attendance has the obvious flaw that a friend can shout “present” for someone who isn’t there. The digital alternatives we have used in college so far each have their own version of the same problem. A Google Form can be filled in from a hostel bed. RFID cards get handed across the table. The Bluetooth-beacon app I tried last semester needed students to keep an extra app open and grant the right Bluetooth permission, and in practice maybe two-thirds of the class did this on any given day.

The starting point for this project was a simpler idea: the WiFi signal from a normal phone hotspot only carries about 8–12 metres indoors, and walls cut it sharply. So if a phone is on the hotspot, the phone is probably in the room. I just had to turn that fact into a check-in with as little fuss as possible.

The result is a single Android app for the teacher. When the teacher starts it, the app first tries to create its own private WiFi hotspot; if the phone or carrier has hotspots disabled, it falls back to using whichever WiFi the phone is already connected to. Either way, an HTTP server starts on port 8080, and a QR code appears on the dashboard. Students scan it, join the WiFi, and the captive page asks for their roll number. From scan to “you’re checked in” takes under fifteen seconds in our tests.

The contributions of the project are: (i) a working classroom attendance system that runs on a teacher’s phone with no backend and no internet; (ii) a privacy model where the stored device identifiers are SHA-256 hashes salted with a per-session UUID, so attendance rows from different days cannot be matched up; and (iii) a two-layer presence check that combines IP-level pings with a browser-level heartbeat, which made it noticeably harder to fake attendance during testing.

2 Related Work

Electronic attendance is not new, and the existing systems mostly fall into three groups. Card-based systems (RFID,

NFC, the older magnetic-stripe ones) need readers installed in every classroom, and even after that you still have to trust that the card in the slot belongs to the person in the seat. Bluetooth-beacon systems put a battery-powered BLE beacon on the wall and ask each student to install a companion app; that removes the reader cost, but it adds a software dependency to every student phone, and the BLE permission flow on Android is honestly a mess — it changed significantly between Android 11, 12, and 13. GPS-fence apps draw a circle around the room’s coordinates and check the student is inside it; they drain battery, they don’t work well in concrete buildings, and a mock-location app from any developer-options guide will defeat them.

Captive-portal check-in is something coffee shops have been doing for years, but I didn’t find a good example of using it for classroom attendance with proper privacy in the academic literature. The nearest thing is institutional WiFi that quietly logs MAC addresses against student IDs at the access point, but that needs deep cooperation from the IT team and exposes the student’s real MAC, both of which I wanted to avoid.

3 System Overview

The whole thing is one Android app with two ways of running. *Hotspot mode* is what we wanted by default — the teacher’s phone uses Android’s local-only hotspot API to put up a private WiFi that exists only while the app is running. But on some phones the carrier disables hotspots, and on others the teacher might already be using mobile data and not want to share it. So the app also has *shared-network mode*, where the teacher just stays on the classroom’s existing WiFi and the HTTP server binds to whatever IP address DHCP gave the phone. The app figures out which mode worked at startup and quietly switches to the other if the first one fails.

Inside the app there are four parts that talk to each other. A hotspot manager that handles the WiFi side and scans the local subnet to see who is connected. A check-in HTTP server (built on NanoHTTPD) that serves the form, validates the roll

[Screenshot 1: Teacher dashboard]

Replace this placeholder with
`\includegraphics`
`[width=\columnwidth]`
`{dashboard.png}`

Figure 1: Teacher dashboard during an active session. The QR code encodes the URL of the embedded HTTP server. Long-pressing the QR panel reveals the underlying URL for manual entry.

number against the roster, and records the check-in. A Room database with four tables — roster, sessions, check-ins, and a presence log used by the attendance calculation. And a foreground service of the data-sync type that keeps everything alive when the teacher's screen locks; without this Android starts killing the process within minutes.

The student's phone is just a generic web client. No companion app to install, no Bluetooth permission to grant, nothing besides joining a WiFi. That was the main UX decision and it shaped a lot of what came later: the system can never ask the student's phone for anything the browser doesn't volunteer, which keeps the trust boundary very small.

4 Privacy-Preserving Identity

There are three kinds of people I worried about while designing this. The first is a teacher who is more curious than they should be about which students attended which classes over the semester. The second is anyone who gets a copy of the database file — a lost phone, a backup that ended up in the wrong cloud account. The third is a pair of students who share a phone and try to mark both of them present from a single device.

For the first two, I rely on rotating the salt every session. At the start of each class the app generates a fresh random salt:

```
fun generateSessionSalt(): String =  
    UUID.randomUUID().toString()
```

Each connected device's IP address is then hashed before any information about it is written to disk:

```
fun anonymize(ip: String, salt: String): String {  
    val input = "${ip.trim()}:$salt"  
    val digest = MessageDigest  
        .getInstance("SHA-256")  
        .digest(input.toByteArray())  
    return digest.joinToString("") {  
        "%02x".format(it)  
    }  
}
```

The 64-character hex result is the only thing that ever gets written to disk. The raw IP only lives in an in-memory map that the hotspot manager wipes the moment a session ends. Because the salt is different every class, the same student

walking in on Monday and Tuesday produces two completely unrelated tokens — even if someone got hold of the whole database, they couldn't match one day's attendance against another.

I wanted to avoid MAC addresses entirely, and as it turns out Android made this decision for me. From Android 10 onwards a normal app simply cannot read MAC addresses any more — the ARP table returns empty to non-system apps, and the WifiInfo accessor returns a hard-coded placeholder. The kernel neighbour table can still be read through the unprivileged IP-neighbour command, and that one does include MACs, but I parse only the IP field and throw the rest of the line away. So even in RAM the app never holds a single real MAC.

The third case — two students sharing one phone — I handle in application logic rather than in the hash. The check-in server enforces both one-check-in-per-token and one-check-in-per-roll-number, so the second attempt from the same phone is rejected with an error message no matter what roll number is typed.

5 Two-Layer Presence Detection

The first version of the app marked a student present the moment they hit submit. That was clearly not enough — a student could type their roll number from the doorway and walk straight back out, and the system would happily record them. So I added a second check that keeps watching for the rest of the class.

Layer 1: subnet pings. Every five seconds during a session the hotspot manager pings every IP on the /24 subnet, with a cap of 32 pings in flight at once so we don't run out of file descriptors. Anything that replies gets put into a record map. Anything we'd seen before but didn't reply this time gets a miss counter bumped; after three misses in a row (about fifteen seconds of silence) we treat the device as gone. There's a second source too — the kernel's neighbour table, read through the IP-neighbour command — because phones in WiFi power-save mode often don't answer ICMP but still show up there.

Layer 2: browser heartbeat. After a student successfully checks in, the success page they land on has a small piece of JavaScript that pings the server every thirty seconds:

```
function sendHeartbeat() {  
    fetch('/heartbeat', {  
        method: 'POST',  
        headers: {'Content-Type': 'application/json'},  
        body: JSON.stringify({  
            token: token, session_id: sid  
        })  
    });  
    sendHeartbeat();  
    setInterval(sendHeartbeat, 30000);  
}
```

Every thirty seconds the browser sends the student's token and the session id. The server updates a row in the presence-log table with the latest timestamp. When we calculate final attendance at the end of the class, a student is counted only if both signals agreed: at least one ping sweep saw their token and at least one heartbeat arrived after the initial check-in.

The two layers cover for each other. Pings alone miss students who join the WiFi but never open the page, and they get fooled by phones that stay on the network after the student

[Screenshot 2: Student check-in flow]

```
Replace this placeholder with
\includegraphics
[width=\columnwidth]
{checkin.png}
```

Figure 2: Student-side experience, captured on a Pixel 6a after joining the teacher’s hotspot. The check-in form and the success page are served by the embedded NanoHTTPD instance over plain HTTP on port 8080.

walks out with the screen off. Heartbeats alone get fooled by a student who closes the tab, and they break completely if the browser has JavaScript turned off. Putting both together makes faking attendance a lot harder — you’d need a phone connected to the WiFi, with the success page tab open, and someone inside the room to actually plant the device. By then it’s easier to just attend the class.

6 Captive Portal Handling

Whenever a phone joins a new WiFi network, the operating system quietly does a small test in the background to figure out whether the network actually reaches the internet. iOS asks Apple’s hotspot-detection URL, Android pings Google’s connectivity-check URL, Windows hits Microsoft’s connectivity-test URL. If the response matches the expected one (HTTP 204 No Content for Android, the literal text “Success” for Apple), the OS quietly moves on. Anything else and the phone shows a “Sign in to WiFi” notification and pops up the captive-portal browser.

What we actually want here is for the OS to leave the student alone. The student is supposed to scan the QR code, not be ambushed by a fake login page that we can’t really build properly anyway. So the check-in server catches every probe URL we know about and returns an empty 204:

```
uri.startsWith("/generate_204") ||
uri.startsWith("/gen_204") ||
uri.startsWith("/ncsi.txt") ||
uri.startsWith("/hotspot-detect") ||
uri.startsWith("/connecttest") ||
uri.startsWith("/success.txt")
-> newFixedLengthResponse(
    Response.Status.NO_CONTENT,
    "text/plain", "")
```

A real captive-portal hijack — intercepting requests to other sites and pointing them at our page — would need either DHCP option 114 or the ability to bind UDP port 53 for DNS redirection. The local-only hotspot API gives us neither. The DHCP server is hidden behind the API and port 53 is already taken by the kernel resolver. So returning 204 is the best we can do, and honestly it works fine in practice — the phone just doesn’t complain, and the student scans the QR.

7 Implementation

The app comes out to roughly 3,200 lines of Kotlin spread across around thirty files. UI is traditional Android Views (I started in Compose, then switched back when I realised how much I needed custom drawing for the QR-card layout). The database is Room with four tables. Async work is done in coroutines on the IO dispatcher. The HTTP server is NanoHTTPD, QR codes come from ZXing, and JSON parsing on the heartbeat is Gson. The crypto is just the JDK’s built-in message-digest and UUID utilities — I deliberately didn’t pull in a third-party crypto library because SHA-256 is not the kind of thing you should be customising for a student project.

Two bugs cost me far more time than they should have, and they’re worth documenting because both came from assumptions that seemed fine until the app met a real network.

The 242-student ghost. My first version of the ping sweep was clever in a bad way — it counted TCP connection-refused replies (RST packets) as “alive” along with real ICMP echoes, on the theory that anything willing to RST a closed port is clearly a real host. The problem was that when the phone’s hotspot wasn’t fully up yet but the cellular data was, the sweep started running over the carrier’s CGNAT subnet, and hundreds of unrelated phones politely RST-ed our probes. One test phone joining showed up in the dashboard as 242 connected students. I dropped the TCP signal completely, locked the scanner to interface names that look like soft access points (the prefixes vary across OEMs), and added a hard cap of 100 live hosts per sweep — if it ever finds more, it throws the whole result out.

Cleartext blocked on Android 9+. The in-app “Test Server” button uses HttpURLConnection to fetch its own diagnostic endpoint. Once I bumped the target SDK, this started failing with a cleartext-not-permitted exception, even though the request was going to the same phone. The fix was a tiny network-security-config XML resource that whitelists cleartext for this app, plus the matching flag in the manifest. I thought about shipping a self-signed TLS cert instead, but installing the trust anchor on every student’s phone would defeat the “no friction” goal we started with.

8 Evaluation

I tested the system in three classroom sessions, with 18, 25 and 31 students out of a 40-student roster. My phone — a Samsung A53 on Android 13 — was the teacher device. Student phones were mostly Samsung A-series and Xiaomi Redmi Note, with four iPhones in the mix. Each session ran the full 50-minute lecture and I kept the app in the background after the initial setup.

Check-in latency. Median time from a student scanning the QR code to the green “you’re checked in” page was about 11 seconds, with the slowest 5% of cases hitting 19 seconds. Almost all of that delay is the WiFi association — once you’re on the network, the actual HTTP round trip is under 80 ms.

False positives. Across all three sessions, no roll number was wrongly marked present. I verified this by doing a visual count at the 30-minute mark and comparing it against the dashboard.

False negatives. Two students were initially recorded as

absent even though they were in the room. Both cases were the same iPhone behaviour — the phone showed the “Sign in to WiFi” notification, the student swiped it away without scanning the QR, and nothing further happened. The check-in form was never opened, so the server never saw them. Adding a more obvious reminder for iPhone users is on the to-do list.

Battery and CPU. The foreground service used about 3.4% of the teacher phone’s battery per hour while actively scanning, which is fine for a 50-minute class. CPU sat at 4–6% on the A53 with small spikes every five seconds when the scan runs. Memory stayed below 90 MB throughout.

9 Discussion and Limitations

WiFi proximity is a useful signal but it isn’t a perfect one. How far the hotspot reaches depends on the chipset, the antenna, and how thick the walls are; in our test classroom the signal still worked about two metres into the corridor, which means a student hanging around just outside the door could in theory still check in. I haven’t yet implemented a received-signal-strength filter, but the per-device round-trip times the scanner already records would let me add one without much trouble.

The system also assumes the student will cooperate at least once, by joining the WiFi and typing their roll number. That’s not going away. Removing this step would mean either tying into a campus-wide MDM identity system (which gives back the cross-class linkability I worked hard to avoid) or making a student-side app (which is exactly the friction the project was meant to remove).

The teacher’s phone being the only server is also a single point of failure. If the teacher’s phone dies mid-class, attendance stops being recorded. The Room database has held up through every crash I’ve forced on it, but a network partition during a heartbeat will silently lose that one heartbeat — I rely on the next 30-second cycle to catch up.

Captive-portal auto-open is wildly inconsistent. It varies between Android versions and is even worse across iOS versions. So I designed the system to not depend on it at all. The QR code is the real entry point; auto-open is a bonus when it happens.

10 Conclusion and Future Work

The main thing this project shows is that a single Android phone, with no extra hardware and no internet, is actually enough to run a real classroom attendance system. The proof of presence comes from the hotspot’s radio range itself, which is something we already know works without having to add new infrastructure. The whole system is built out of standard Android APIs, an embedded HTTP server, and the basic crypto that ships with the JDK. I’d like for it to be a useful starting point for someone else building a minimal-trust attendance tool.

There are three things I’d like to add next. The first is fusing WiFi RSSI with passive BLE scans to get a tighter idea of how close a phone actually is, which would address the corridor problem. The second is optional encrypted backup of the Room database to the teacher’s own cloud — since the per-session salts already mean the database is safe to upload, this is mostly a UX exercise. The third is an instructor-side analytics screen showing per-student attendance trends across ses-

sions, which is a bit harder because doing this without breaking cross-session unlinkability requires the analytics to compute on hashed tokens without ever joining them with rosters at rest.

Acknowledgements

Thanks to the students who patiently joined the hotspot during my test sessions, and to the classmates who spotted the 242 ghost-devices bug within minutes of me deploying it.

References

- [1] Android Developers. *WifiManager.startLocalOnlyHotspot reference documentation*. <https://developer.android.com/reference/android/net/wifi/WifiManager>
- [2] Android Developers. *Network security configuration*. <https://developer.android.com/training/articles/security-config>
- [3] Y. Rekhter, B. Moskowitz, D. Karrenberg, G. de Groot, and E. Lear. *Address Allocation for Private Internets*. RFC 1918, IETF, February 1996.
- [4] W. Kumari, O. Gudmundsson, P. Ebersman, and S. Sheng. *Captive-Portal Identification Using DHCP or Router Advertisements*. RFC 7710, IETF, December 2015.
- [5] NanoHTTPD project. *nanohttpd: Tiny, easily embeddable HTTP server in Java*. <https://github.com/NanoHttpd/nanohttpd>
- [6] ZXing project. *ZXing: Barcode image processing library*. <https://github.com/zxing/zxing>
- [7] Android Developers. *Room persistence library*. <https://developer.android.com/training/data-storage/room>
- [8] NIST. *FIPS PUB 180-4: Secure Hash Standard (SHS)*. NIST, August 2015.
- [9] M. K. M. Rabbi and S. Hossain. *Bluetooth Low Energy Based Smart Attendance System*. International Journal of Computer Applications, 174, 2021.
- [10] A. Patel, S. Bhalerao, and N. Ravat. *RFID-based attendance management system*. International Journal of Engineering and Technology, 9, 2017.
- [11] Android Open Source Project. *MAC address randomization in Android 10 and higher*. <https://source.android.com/docs/core/connect/wifi-mac-randomization-behavior>